📚 Jednosměrně Vázaný Seznam (Singly Linked List) – Seznam Úkolů k Maturitě

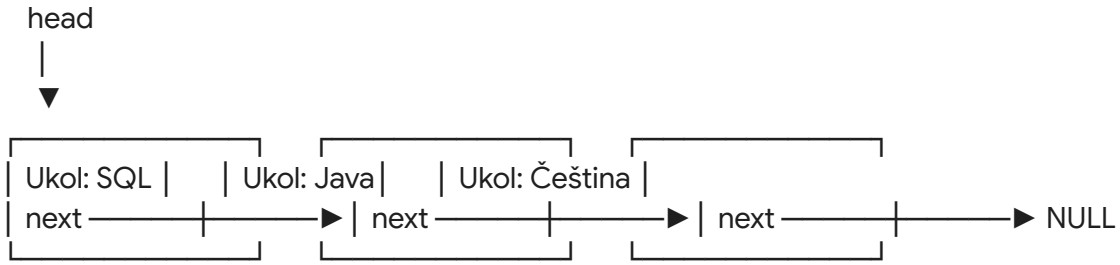
Tento projekt implementuje vlastní **jednosměrně vázaný spojový seznam** v jazyce Java bez použití vestavěných datových struktur z java.util. Slouží jako praktický příklad pro správu seznamu maturitních úkolů s prioritami.

🧩 Jak funguje spojový seznam?

Na rozdíl od klasického pole (Array nebo ArrayList), kde prvky leží souvisle vedle sebe v paměti, jsou prvky ve spojovém seznamu v paměti **rozptýlené**.

Spojový seznam si můžeš představit jako **vláček nebo řetěz lidí držíce se za ruce**:

- Každý článek drží své vlastní informace a odkaz (ukazatel) na **další** článek v řadě.
- Seznam drží pouze odkaz na první prvek – **hlavičku (head)**.



🏗️ Stavební kameny architektury

Struktura se skládá ze tří hlavních tříd:

1. **Ukol (Datový objekt):**
   - Obsahuje samotná data – text úkolu a jeho prioritu (1 až 4).
   - Vůbec netuší, že je součástí nějaké datové struktury.
2. **Node (Uzel / Článek řetězu):**
   - Obaluje objekt Ukol.
   - Obsahuje odkaz next směřující na následující uzel v seznamu.
3. **MyLinkedList (Správce seznamu):**
   - Uchovává ukazatel head (začátek seznamu).
   - Poskytuje metody pro manipulaci s prvkem (přidávání, mazání, vyhledávání, obrácení).

⚡ Základní operace a jejich fungování

1. Přidání na začátek (addFirst) — $O(1)$

- **Princip:** Vytvoří se nový uzel, jeho next se nasměruje na stávající head a head se přenastaví na tento nový uzel.
- **Výhoda:** Blesková operace, není potřeba nic procházet ani posouvat v paměti.

2. Přidání na konec (add) — $O(n)$

- **Princip:** Jelikož seznam zná přímo jen head, musí projít od prvního uzlu po ukazatelích next až na konec (current.next == null). Tomuto poslednímu uzlu nastaví next na nový uzel.

3. Vkládání a mazání uprostřed (addToIndex, removeOnIndex)

- **Princip:** Na rozdíl od pole se **žádné prvky v paměti neposouvají**. Seznam pouze přepojí ukazatele:
  - **Při smazání:** Předchozí uzel obchází mazaný prvek a ukazuje rovnou na následující (current.next = current.next.next).
  - Mazaný prvek zůstane bez odkazů a Java Garbage Collector ho automatiquement odstraní z paměti.

4. Obrácení seznamu (flipList) — $O(n)$

- **Princip:** Pozice uzlů v paměti se nemění. Cyklem se projde celý seznam a u každého uzlu se **otočí směr ukazatele next** směrem k předchozímu uzlu (prev).

🔄 Porovnání: LinkedList vs ArrayList / Pole

Operace / Vlastnost	Spojový seznam (LinkedList)	Pole / ArrayList
Přístup k prvku na indexu	Pomalý $O(n)$ *(musí se projít od začátku)*	Okamžitý $O(1)$ *(přímá adresa)*
Vložení/Smazání na začátku	Bleskové $O(1)$ *(přepojení odkazu)*	Pomalé $O(n)$ *(musí se posunout vše)*
Vložení/Smazání uprostřed	Rychlé přepojení $O(1)$ *(po vyhledání)*	Pomalé $O(n)$ *(nutný posun prvků)*
Využití paměti	Vyšší *(ukládá navíc odkazy next)*	Efektivní *(pouze samotná data)*
Velikost v paměti	Dynamická *(rostoucí dle potřeby)*	Pevná nebo s přealokací

💻 Přehled dostupných metod v projektu

- add(Ukol val) – Přidání prvku na konec
- printAll() – Výpis všech prvků v pořadí
- addFirst(Ukol val) – Přidání prvku na začátek
- removeFirst() – Odstranění prvního prvku
- removeOnIndex(int index) – Odstranění prvku na konkrétním indexu
- addToIndex(Ukol val, int index) – Vložení prvku na specifikovanou pozici
- printAllWithPriority(int priority) – Filtrovaný výpis podle priority (1–4)
- removeNodeWithName(String name) – Vymazání úkolu podle přesného názvu
- flipList() – Obrácení celého seznamu